

Trabajo Práctico Final

Codificación FEC, Diseño VLSI e Implementación en FPGA

Transmisor y receptor BCH concatenado a 800 Gb/s

Alumno	Damián Lugano
Docente	Agustín Pesce
Institución	Fundación Fulgor, Córdoba, Argentina
Plataforma	Xilinx Zynq UltraScale+ RFSoc XCZU48DR (ZCU208)
Toolchain	Python / NumPy, Synopsys VCS, Synopsys Design Compiler, AMD Vivado
Versión	1.0

Resumen del trabajo

Se debe diseñar, modelar, implementar en RTL, verificar y sintetizar una cadena completa de transmisión con corrección de errores hacia adelante (FEC) operando a 800 Gb/s de tasa de línea. El trabajo abarca las tres materias en forma integrada: la teoría de códigos algebraicos define el algoritmo, el diseño VLSI define cómo paralelizarlo para alcanzar la frecuencia objetivo, y la FPGA provee la validación funcional del RTL sobre silicio real.

Modalidad. El trabajo se desarrolla en siete etapas secuenciales. Cada etapa tiene entregables y criterios de aceptación explícitos. **Una etapa no se da por aprobada hasta que cumple su criterio de aceptación**, y no se debe avanzar a la siguiente sin haberla cerrado. Esta secuencia no es burocracia: refleja el orden real en que se construye un chip, donde cada nivel de abstracción se valida antes de bajar al siguiente.

Sobre las notas al pie de cada etapa. Varias etapas incluyen una nota que prohíbe explícitamente cierta implementación o que señala un fenómeno a investigar. Esas notas no son sugerencias: marcan los puntos donde la solución obvia es la incorrecta, y su justificación forma parte de la evaluación.

1 Objetivos

Al finalizar el trabajo el alumno debe ser capaz de:

- Derivar los parámetros de un código BCH primitivo sobre un campo de Galois y justificar por qué ciertas combinaciones (n, k) son realizables y otras no.
- Traducir un algoritmo secuencial sobre $GF(2^m)$ a una arquitectura paralela capaz de sostener cientos de gigabits por segundo.
- Dimensionar una cadena de datos multi-dominio: calcular anchos de bus, frecuencias, profundidades de FIFO y latencias a partir de un requisito de tasa.
- Implementar RTL sintetizable con restricciones de estilo industriales y cerrar *timing* a una frecuencia objetivo agresiva.
- Construir un banco de pruebas orientado a objetos en SystemVerilog con modelo de referencia, *scoreboard* y medición de cobertura.
- Validar funcionalmente el diseño sobre FPGA y producir curvas de BER medidas en hardware.

2 Especificación del sistema

2.1 Arquitectura

La cadena completa a implementar se muestra en la Figura 1. El bloque de canal existe únicamente en el entorno de verificación y en la FPGA; no forma parte del diseño que se sintetiza para ASIC.

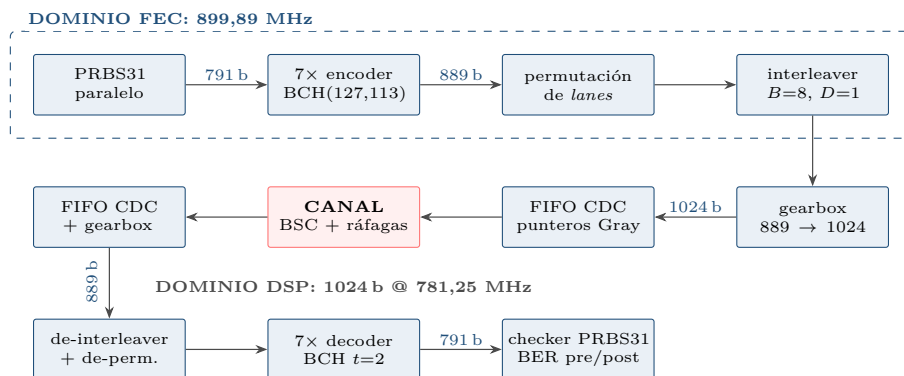


Figura 1: Cadena completa TX, canal y RX. El receptor replica el paralelismo del transmisor. Las etiquetas indican el ancho del bus donde cambia.

2.2 Parámetros fijos

Los siguientes valores son **especificación** y no pueden modificarse. Varios de ellos son consecuencia de decisiones de arquitectura que el alumno deberá justificar en el informe.

Parámetro	Valor	Observación
Código	BCH(127,113)	primitivo, banda angosta, sobre $GF(2^7)$
Polinomio primitivo	$x^7 + x^3 + 1$	0x89
Capacidad de corrección	$t = 2$	14 bits de paridad
Tasa del código	$113/127 = 0,8898$	overhead 12,39 %
Decisión	dura (<i>hard</i>)	el decodificador no recibe confiabilidades
Encoders en paralelo	7	a derivar en la Etapa 0
Bus dominio FEC	791 b entrada / 889 b salida	
Frecuencia FEC	899,89 MHz	objetivo de síntesis: cerrar > 800 MHz
Bus dominio DSP	1024 b	

Parámetro	Valor	Observación
Frecuencia DSP	781,25 MHz	ratio exacto 889:1024, gcd = 1
Tasa de línea	800,000 Gb/s	medida a la salida del DSP
Tasa de payload	711,811 Gb/s	
Interleaver	convolucional (Forney), $B=8, D=1$	
Secuencia de prueba	PRBS31: $x^{31} + x^{28} + 1$	

Advertencia sobre el código elegido

El código BCH(127,113) fue especificado deliberadamente. Antes de implementar nada, el alumno debe verificar que los parámetros son realizables y estar en condiciones de explicar, en el informe, por qué otras combinaciones (n, k) con $n = 127$ que parecen razonables a primera vista son en realidad imposibles de construir. La justificación debe apoyarse en la estructura de las clases ciclotómicas módulo 127 sobre GF(2).

3 El interleaver: mecanismo y propósito

Esta sección explica *cómo funciona* el esquema de entrelazado especificado. Las derivaciones cuantitativas (en particular la longitud máxima de ráfaga corregible) son parte del trabajo a realizar y se plantean como preguntas en la Sección 6.

3.1 Por qué hace falta

Un código BCH(127, 113) con $t = 2$ corrige **hasta dos errores por palabra de código**, sin importar dónde estén. Esa garantía es por codeword, no global: tres errores que caen en el mismo codeword son incorregibles, mientras que doscientos errores repartidos de a uno en doscientos codewords distintos se corrigen todos.

Un canal real no reparte los errores de manera uniforme. Una perturbación física (un evento de ruido impulsivo, una pérdida momentánea de *lock* en el CDR, una interferencia) afecta *bits consecutivos de la línea*. Eso es una **ráfaga**, y es el peor caso posible para un código algebraico: concentra todos los errores donde más daño hacen.

El interleaver resuelve esto separando en el tiempo bits que pertenecen al mismo codeword. La idea es simple: si los bits de un codeword viajan por la línea muy alejados entre sí, entonces bits vecinos en la línea pertenecen a codewords distintos, y una ráfaga se reparte de a un error por codeword en lugar de concentrarse.

La regla de oro

El interleaver no corrige ni un solo bit. Lo único que hace es **redistribuir** los errores para que el decodificador los vea dentro de su capacidad de corrección. Toda la corrección la hace el BCH; el interleaver solo se asegura de que le lleguen los errores de a poco.

El esquema especificado tiene **dos etapas**. La primera es puramente combinatorial y no cuesta nada; la segunda usa registros. Se aplican en ese orden.

3.2 Etapa 1: permutación de *lanes*

Cada ciclo del dominio FEC se producen 7 codewords en paralelo, uno por cada encoder. La pregunta es en qué orden ponerlos en la línea de 889 bits.

La opción ingenua es concatenarlos: primero los 127 bits del codeword 0, después los del codeword 1, y así. Eso es lo peor que se puede hacer, porque deja 127 bits consecutivos de la línea perteneciendo todos al mismo codeword.

La permutación especificada hace lo contrario: **intercala** los codewords bit a bit. El bit j de todos los codewords sale junto, y después el bit $j + 1$:

$$\text{línea}[7j + \ell] = \text{CW}_\ell[j], \quad \ell \in \{0, \dots, 6\}, \quad j \in \{0, \dots, 126\} \quad (1)$$

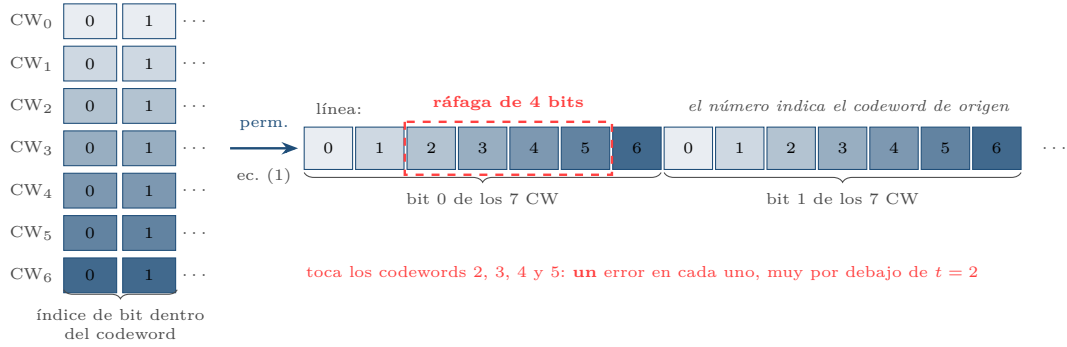


Figura 2: Permutación de *lanes*. Al intercalar los siete codewords bit a bit, una ráfaga corta de la línea toca un bit distinto de cada codeword en lugar de concentrarse en uno solo. El color y el número dentro de cada celda identifican el codeword de origen.

El efecto se ve en la Figura 2: una ráfaga de 4 bits consecutivos en la línea pega en 4 codewords distintos, con *un* error en cada uno. Cada decodificador ve un error, que está muy por debajo de su capacidad $t = 2$.

En RTL esta etapa es **cableado puro**. No hay registros, no hay lógica, no hay latencia: es un reordenamiento de índices que se escribe con **assign** y que el sintetizador resuelve como renombrado de señales. Cuesta exactamente cero área y cero timing.

3.3 Etapa 2: interleaver convolucional de Forney

La permutación de *lanes* protege contra ráfagas cortas, pero su alcance está limitado por el número de codewords en paralelo. Para ráfagas más largas hace falta separar los bits también **a lo largo del tiempo**, y de eso se encarga el interleaver convolucional.

El mecanismo es el siguiente. Las posiciones de bit de la línea se reparten cíclicamente entre B ramas. Cada rama aplica un retardo distinto, múltiplo de D :

$$\text{rama}(p) = p \bmod B, \quad \text{retardo}_{\text{TX}}(p) = (p \bmod B) \cdot D \quad (2)$$

La rama 0 pasa derecho sin retardo, la rama 1 se retrasa D ciclos, la rama 2 se retrasa $2D$, y así hasta la rama $B - 1$ que se retrasa $(B - 1)D$. El resultado es que bits que entraron *juntos* salen *escalonados* en el tiempo.

Por qué el de-interleaver es el espejo

El receptor debe deshacer el escalonamiento, y lo hace aplicando el retardo complementario:

$$\text{retardo}_{\text{RX}}(p) = (B - 1 - (p \bmod B)) \cdot D \quad (3)$$

Sumando (2) y (3), todo bit acumula $(p \bmod B)D + (B - 1 - (p \bmod B))D = (B - 1)D$ ciclos, **independientemente de su posición**. Esa es la propiedad clave: la latencia total es *uniforme*, así que el alineamiento de los codewords se preserva sin necesidad de ninguna lógica de realineado. Con $B = 8$ y $D = 1$ la latencia es de 7 ciclos por lado.

Costo en hardware

La memoria total de un interleaver convolucional es la suma de los retardos de todas las ramas. Por lado:

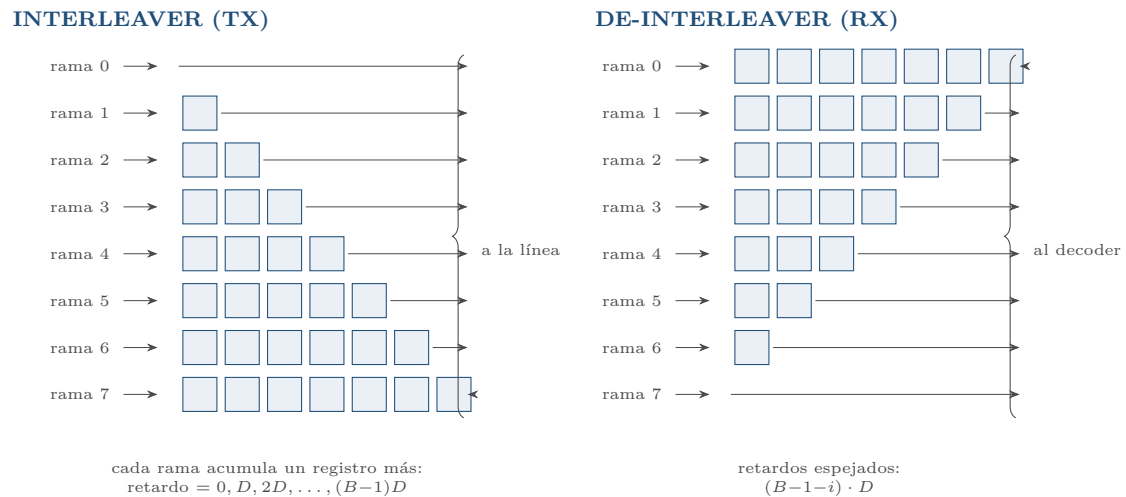


Figura 3: Interleaver convolutivo de Forney con $B = 8$ ramas. El de-interleaver aplica los retardos complementarios, de modo que *todo* bit acumula exactamente $(B - 1)D$ ciclos entre los dos lados.

$$M_{\text{lado}} = \sum_{i=0}^{B-1} i \cdot D \cdot \frac{W}{B} = \frac{B(B-1)}{2} \cdot D \cdot \frac{W}{B} = \frac{(B-1) \cdot D \cdot W}{2} \quad (4)$$

donde $W = 889$ es el ancho de la línea. Nótese que la memoria crece **linealmente** con B pero la protección también, así que el compromiso es directo. Con $B = 8$ y $D = 1$ resultan unos pocos miles de bits por lado: ruido estadístico en el reporte de área.

En FPGA estos retardos escalonados mapean de forma natural a las primitivas SRL32 del *fabric*, que existen precisamente para esto. En ASIC, con B chico son registros; con B grande conviene evaluar memorias.

Comparación con el interleaver de bloque

El alumno debe poder explicar, en el informe, por qué se eligió un interleaver convolutivo en lugar de uno de bloque. Los ejes de comparación son: memoria requerida para igual protección, latencia, necesidad de generación de direcciones, necesidad de memoria *ping-pong*, y tamaño del multiplexor de lectura. Este último punto es relevante para el cierre de timing.

3.4 Cómo interactúan las dos etapas

Acá está el punto conceptualmente más interesante del diseño, y el que el alumno debe analizar con cuidado.

Las dos etapas usan módulos distintos: la permutación reparte los bits entre **7** codewords, y el Forney los reparte entre **8** ramas. Dos bits terminan en el mismo codeword *y* en la misma rama solo si coinciden en ambos módulos a la vez.

La estructura aritmética que surge de combinar ambos módulos es lo que determina la protección real del sistema, y esa protección es considerablemente mayor que la de cualquiera de las dos etapas por separado. **Derivar cuantitativamente ese resultado es parte del trabajo** (pregunta P4 del informe).

Hay además un segundo efecto, más sutil y fácil de pasar por alto. La asignación de rama de la ecuación (2) se calcula sobre la posición *p* dentro del frame de 889 bits. Como 889 no es múltiplo de $B = 8$, la correspondencia entre posición absoluta en la línea y rama asignada **se corre al cruzar de un frame al siguiente**. El alumno debe analizar qué consecuencia tiene esto sobre una ráfaga que cae justo en el borde entre dos frames (pregunta P5).

Advertencia metodológica

Al medir experimentalmente la longitud máxima de ráfaga corregible, no alcanza con probar ráfagas en posiciones arbitrarias o centradas dentro del frame. Es necesario **barrer todos los offsets de arranque posibles** dentro de un período completo. Una campaña de medición que no cubra los bordes de frame va a reportar un límite optimista, y la conclusión será incorrecta.

3.5 Interacción con el control de flujo

Todos los registros de retardo del interleaver y del de-interleaver deben tener *clock enable*. Cuando la FIFO aplica *backpressure*, la cadena entera se congela en lugar de descartar datos: el *enable* se propaga hacia atrás y todo el pipeline (PRBS, encoders, interleaver) se detiene en el mismo ciclo. Esto es lo que permite frenar el generador sin perder un solo bit ni corromper el alineamiento del entrelazado.

4 Etapas del trabajo

Cada etapa lista sus entregables y su criterio de aceptación. El criterio de aceptación es binario: se cumple o no se cumple. No se admiten resultados parciales para avanzar.

Etapa 0: Análisis y dimensionamiento

Objetivo. Establecer, sobre papel y con cuentas verificables, todos los números del sistema antes de escribir una sola línea de código.

Entregables

- Memoria de cálculo justificando el número de encoders paralelos a partir de la relación $\text{tasa} = N \cdot 127 \cdot f_{\text{fec}}$, incluyendo las alternativas descartadas y por qué.
- Demostración de que BCH(127,113) es realizable y de qué otros (n, k) con $n = 127$ lo son.
- Cálculo del polinomio generador $g(x)$ como producto de polinomios minimales, con su grado.
- Dimensionamiento de la FIFO CDC: profundidad, umbral de arranque y justificación.
- Presupuesto de latencia *end-to-end* en ciclos y en nanosegundos, incluyendo el aporte del interleaver según la Sección 3.

Criterio de aceptación

- ✓ Todos los números cierran con la tasa objetivo de 800,000 Gb/s con error menor al 0,1 %.
- ✓ El grado de $g(x)$ calculado coincide con los 14 bits de paridad especificados.

Nota

El ratio 889:1024 tiene $\text{gcd} = 1$. El alumno debe analizar qué implica esto para el patrón de llenado de la FIFO y por qué el arranque del lado de lectura no puede hacerse con la FIFO vacía.

Etapa 1: Modelo de referencia en Python

Objetivo. Construir un modelo bit-exacto de la cadena completa. Ningún RTL puede escribirse antes de cerrar esta etapa.

Entregables

- Aritmética de $\text{GF}(2^7)$: tablas exp/log, multiplicación, inverso, traza.

- Generación del polinomio generador y de la matriz de codificación paralela.
- Codificador y decodificador BCH(127,113) $t = 2$ con decisión dura.
- PRBS31 paralelizado a 791 bits mediante potenciación de la matriz de transición.
- Permutación de *lanes* e interleaver convolucional, ambos con su inversa.
- Modelo de canal: BSC con probabilidad p configurable, y generador de ráfagas de longitud parametrizable.
- Suite de autotests y rutinas de barrido de BER.

Criterio de aceptación

- ✓ Síndromes nulos para todo codeword válido generado aleatoriamente.
- ✓ Corrección correcta y completa de 0, 1 y 2 errores en posiciones arbitrarias.
- ✓ El PRBS31 paralelo de 791 bits coincide bit a bit con la implementación serie.
- ✓ Ambos interleavers son exactamente reversibles, con latencia uniforme para todo bit.
- ✓ La cadena completa sin ruido produce BER = 0.

Nota

El decodificador debe implementarse **sin búsqueda de Chien**. Para $t = 2$ el polinomio localizador es cuadrático y admite solución cerrada en el campo. El alumno debe investigar cómo resolver $x^2 + x = c$ en $\text{GF}(2^m)$ con m impar, y justificar cuantitativamente en el informe la diferencia de área y de camino crítico respecto de instanciar 127 evaluadores.

Etapla 2: RTL del transmisor

Objetivo. Implementar en SystemVerilog sintetizable la cadena de transmisión completa.

Entregables

- Generador PRBS31 paralelo de 791 bits con *clock enable*.
- Banco de 7 codificadores BCH(127,113) paralelos.
- Permutación de *lanes*.
- Interleaver convolucional $B=8$, $D=1$ con *clock enable* en todos los registros de retardo.
- Gearbox 889 a 1024 bits.
- FIFO CDC con punteros Gray, profundidad y arranque según Etapa 0.

Criterio de aceptación

- ✓ Simulación contra los vectores dorados de la Etapa 1 sin una sola discrepancia.
- ✓ Cero *latches* inferidos y cero *warnings* de síntesis relevantes.
- ✓ El diseño respeta las restricciones de estilo de la Sección 5.

Etapla 3: RTL del receptor

Objetivo. Implementar el camino inverso completo, incluyendo el decodificador algebraico.

Entregables

- FIFO CDC de retorno y gearbox 1024 a 889 bits.
- De-interleaver y de-permutación de *lanes*.
- Banco de 7 decodificadores BCH $t = 2$: cálculo de síndromes S_1 y S_3 , resolución del localizador y corrección.

- Checker PRBS31 con contadores de BER pre-FEC y post-FEC.

Criterio de aceptación

- ✓ El lazo completo de TX a RX reproduce exactamente el resultado del modelo de Python para las mismas semillas y el mismo patrón de errores.
- ✓ Los contadores de BER coinciden con los del modelo dentro del error estadístico.

Etapa 4: Verificación en SystemVerilog

Objetivo. Construir un banco de pruebas orientado a objetos, con modelo de referencia y *scoreboard* automático.

Entregables

- Clase de transacción con los campos relevantes de la cadena.
- Generador de estímulos con control de semilla, tasa de error y longitud de ráfaga.
- *Driver* y monitores de entrada y salida.
- *Scoreboard* que compara contra los vectores dorados producidos por el modelo de Python.
- Modelo de cobertura funcional: número de errores por codeword (0, 1, 2, 3 o más), longitud de ráfaga, ocupación de la FIFO, condiciones de *backpressure*.
- Suite de tests dirigidos y aleatorios con reporte automático.

Criterio de aceptación

- ✓ Cobertura funcional mayor o igual al 95 % sobre el modelo definido.
- ✓ Cobertura de código (líneas, ramas, condiciones) mayor o igual al 90 %.
- ✓ Todos los tests pasan con al menos tres semillas distintas.
- ✓ El *scoreboard* no contiene una reimplementación del algoritmo BCH: consume los vectores del modelo de Python.

Nota

No se debe usar UVM. Para el alcance de este trabajo, una infraestructura propia por capas alcanza y permite dedicar el tiempo al diseño en lugar de a la configuración del *framework*.

Etapa 5: Síntesis lógica

Objetivo. Cerrar *timing* en el dominio FEC a la frecuencia objetivo y reportar área.

Entregables

- Scripts de Design Compiler con restricciones completas: relojes, grupos de excepciones, entradas y salidas.
- Restricciones de CDC correctamente declaradas sobre la FIFO.
- Reportes de *timing*, área y potencia.
- Análisis del camino crítico con identificación del bloque responsable.

Criterio de aceptación

- ✓ El dominio FEC cierra con *slack* no negativo a una frecuencia superior a 800 MHz.
- ✓ Cero violaciones de DRC.
- ✓ Los cruces de dominio están declarados y no aparecen como caminos con *timing* real.

Nota

Aunque los dos relojes se derivan del mismo PLL, la relación de fase entre ellos es desconocida. El alumno debe determinar si eso obliga o no a tratar el cruce como CDC y justificar la respuesta.

Etapa 6: Implementación y medición en FPGA

Objetivo. Validar funcionalmente el RTL sobre hardware y producir curvas de BER medidas.

Entregables

- *Wrapper* de *clocking* que deriva ambos relojes desde un único MMCM manteniendo la relación 889:1024, a una frecuencia reducida.
- Generador de ruido en *fabric*: canal binario simétrico con probabilidad p ajustable en tiempo de ejecución.
- Generador de ráfagas con longitud ajustable.
- Interfaz de control y lectura de contadores (AXI-Lite hacia el PS, o equivalente).
- Reporte de utilización de recursos e informe de *timing* de Vivado.
- Curvas medidas: BER post-FEC contra BER pre-FEC, y BER post-FEC contra longitud de ráfaga.
- Capturas de pantalla de las mediciones.

Criterio de aceptación

- ✓ La BER medida en FPGA coincide con la del modelo de Python dentro del error estadístico.
- ✓ La curva de ráfagas exhibe un umbral neto: por debajo de cierta longitud la corrección es total, por encima el sistema colapsa.
- ✓ El barrido de longitud de ráfaga cubre todos los offsets de arranque dentro del frame, no solamente posiciones centradas.
- ✓ El diseño cierra *timing* en Vivado a la frecuencia reducida elegida.

Nota

El generador de ruido **no debe** ser un generador gaussiano. El alumno debe justificar por qué, dado que el decodificador es de decisión dura, un modelo de canal más simple es exactamente equivalente y por qué el AWGN sería área desperdiciada. La conversión a E_b/N_0 para los gráficos se hace en post-procesamiento.

5 Restricciones de estilo de RTL

El RTL se evalúa además de por su funcionalidad, por su calidad como código de diseño industrial. Las siguientes restricciones son obligatorias.

- **RTL plano y explícito.** Usar `assign`, `always_comb` y `always_ff`. Evitar `function`, `task` y bucles generativos donde la escritura explícita sea posible. Las matrices constantes derivadas en Python se vuelcan como asignaciones explícitas.
- **Sin lógica combinacional en los límites de módulo** que cruce dominios de reloj.
- **Un único estilo de reset** en todo el diseño, declarado y justificado.
- **Parametrización real.** El número de encoders, B y D del interleaver, y la profundidad de la FIFO deben ser parámetros, no constantes embebidas.
- **Sin inferencia accidental de *latches*.** Toda rama combinacional asigna todas sus salidas.

- **Clock enable consistente.** Todo registro del pipeline de datos debe respetar el *enable* de *backpressure*.
- **Nombres consistentes** y separación clara entre señales registradas y combinacionales.

6 Informe

El informe debe incluir, como mínimo, las siguientes discusiones. Estas preguntas no son opcionales: son el núcleo conceptual del trabajo y su respuesta se evalúa.

- P1.** ¿Por qué ciertos pares (n, k) con $n = 127$ no son realizables como código cíclico binario? ¿Qué valores de $n - k$ sí lo son y por qué?
- P2.** ¿Cómo se deriva el número de codificadores paralelos a partir del requisito de tasa y del piso de frecuencia? ¿Qué alternativas se descartaron y con qué criterio?
- P3.** ¿Cómo se resuelve el polinomio localizador para $t = 2$ sin búsqueda de Chien? Cuantificar la diferencia de área y de camino crítico respecto de la implementación con 127 evaluadores.
- P4.** La permutación de *lanes* reparte los bits entre 7 codewords y el interleaver convolucional entre 8 ramas. ¿Qué propiedad aritmética de los números 7 y 8 hace que la combinación proteja mucho más que cada etapa por separado? Derivar analíticamente la longitud máxima de ráfaga corregible y verificarla experimentalmente.
- P5.** El ancho del frame de línea es 889 bits, que no es múltiplo de $B = 8$. Analizar qué ocurre con una ráfaga que cruza el borde entre dos frames consecutivos. ¿La longitud máxima corregible sigue siendo la misma que dentro de un frame? Verificar la respuesta barriando *todos* los offsets de arranque posibles dentro de un frame, y explicar el resultado con la aritmética modular correspondiente.
- P6.** Si el resultado de la pregunta anterior resulta ser peor que el de la pregunta P4, proponer una modificación del RTL que recupere la protección completa. Estimar su costo en área y en camino crítico, y argumentar si conviene o no implementarla.
- P7.** Comparar el interleaver convolucional con uno de bloque de protección equivalente: memoria, latencia, generación de direcciones y tamaño del multiplexor de lectura. Justificar la elección.
- P8.** Con ratio de tasa exacto entre ambos dominios, ¿por qué la FIFO no puede arrancar vacía? ¿Qué mecanismo de arranque se implementó?
- P9.** ¿A partir de qué tasa de error de entrada el FEC deja de mejorar la BER y empieza a empeorarla? Explicar el mecanismo de miscorrección y estimar la fracción de codewords con 3 errores que se miscorrigen en silencio.
- P10. Pregunta de cierre.** Este diseño toma tres decisiones que fijan su lugar en el espacio de compromisos del control de errores: un código de *rate* alto (0,89), decisión dura, y un entrelazado corto. Cada una de las tres resigna capacidad de corrección a cambio de otra cosa. Identificar qué se gana con cada decisión y estimar, aunque sea en orden de magnitud, cuánta ganancia de codificación se paga por ellas. Discutir después por qué las tres son coherentes entre sí, es decir, por qué no tendría sentido combinar decisión dura con un código de *rate* muy bajo, ni decisión blanda con un entrelazado de siete ciclos. Finalmente: si el requisito cambiara y hubiera que bajar la BER de salida dos órdenes de magnitud manteniendo la tasa de línea, ¿cuál de las tres decisiones conviene revisar primero, y qué costo tendría en área, latencia y frecuencia?

7 Evaluación

Componente	Peso	Qué se evalúa
Etapas 0 y 1: análisis y modelo	20 %	Corrección de las derivaciones; completitud de los auto-tests
Etapas 2 y 3: RTL	25 %	Funcionalidad, calidad del código, adhesión a las restricciones de estilo
Etapas 4: verificación	20 %	Arquitectura del banco de pruebas, cobertura alcanzada, independencia del <i>scoreboard</i>
Etapas 5: síntesis	15 %	Frecuencia alcanzada, manejo correcto de CDC, análisis del camino crítico
Etapas 6: FPGA	10 %	Funcionamiento en hardware, calidad de las mediciones
Informe	10 %	Profundidad de las respuestas de la Sección 6

Criterio general

Se valora más un diseño que alcanza una frecuencia algo menor pero cuyo camino crítico está correctamente identificado y explicado, que uno que cierra *timing* sin que el alumno sepa por qué. Lo mismo aplica a los resultados de BER: una curva acompañada de una explicación del mecanismo físico que la produce vale más que una curva sin discusión.